

Raspodijeljeni sustavi, Predavanje 04

Nazivlje i otkrivanje usluga

Fakultet informatike, Sveučilište Jurja Dobrile u Puli

izv. prof. dr. sc. Nikola Tanković

1 Ishodi učenja

Nakon ovog predavanja trebali biste moći:

1. Razlikovati ime, adresu i identifikator te objasniti zašto klijent poziva uslugu po imenu, a ne po adresi pojedine instance.
2. Opisati razrješavanje imena u DNS-u: stablo imena, resolver i poslužitelje imena.
3. Objasniti predmemoriranje (caching) i TTL te procijeniti koliko dugo zapis u predmemoriji može biti zastario.
4. Usporediti otkrivanje usluga na klijentu i preko usmjernika te samoregistraciju i registraciju koju obavlja treća strana.
5. Objasniti najam (lease) i kompromis između trajanja najma, učestalosti obnove i lažnog uklanjanja ispravne instance.
6. Primijeniti imena usluga u Docker Composeu, Docker Swarmu i gRPC-u.

Predznanje: predavanje 02 (granica usluge, replike, Docker Compose i Swarm, granice razmještaja) i predavanje 03 (gRPC kanal `predaja:50051`, istek čekanja, ponovni pokušaj, idempotentnost).

Opseg predavanja: imena, adrese i identifikatori, DNS, predmemoriranje i TTL, otkrivanje usluga, registracija i najmovi. Sigurnost imena i certifikati obrađuju se u predavanju 12, usklađivanje kopija registra u predavanjima 06 i 07, otkrivanje ispada u predavanju 08, a raspodjela opterećenja u predavanju 11. Portal za predaju projekata i dalje je zamišljeni primjer, a ne stvarni sustav sveučilišta.

2 Primjer: gdje je usluga predaje?

U predavanju 03 portal je otvarao gRPC kanal prema adresi `predaja:50051`. Riječ `predaja` nije IP adresa, nego **ime** usluge. Netko ga mora pretvoriti u adresu računala na kojemu usluga stvarno radi.

To nije jednokratan posao. Najprije se podsjetimo odakle dolaze kontejneri (Figure 1). Iz koda i datoteke `Dockerfile` naredba `docker build` gradi **sliku** (image): predložak samo za čitanje s uputama za stvaranje kontejnera. **Kontejner** je pokrenuta instanca slike [12]. Sliku spremamo u **registar slika** naredbom `docker push`, a naredbe `docker pull` i `docker run` je odande dohvaćaju; Docker Hub je javni registar slika [12]. Iz jedne slike može nastati mnogo kontejnera. Riječ „registar" ovdje znači spremište slika; registar usluga, koji uvodimo kasnije, čuva adrese kontejnera koji rade.

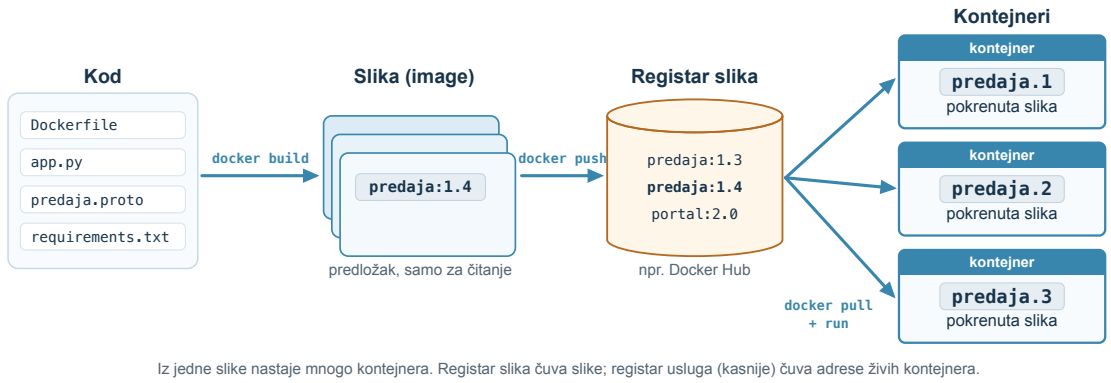


Figure 1: Od koda do kontejnera: docker build gradi sliku, docker push je sprema u registar slika, a docker pull i run iz nje pokreću kontejnere. Iz jedne slike nastaje više kontejnera.

Pred rok usluga predaje radi u tri replike na tri čvora Docker Swarma (predavanje 02). Kada stvaramo uslugu, navodimo koju sliku koristi [11]. U Swarmu zadajemo **željeno stanje** usluge: usluga predaje treba raditi u tri kontejnera iste slike. **Upravitelj** (manager) to stanje čuva, raspoređuje **zadatke** (tasks) na radne čvorove i stalno uspoređuje stvarno stanje sa željenim; svaki zadatak je jedan kontejner s vlastitom adresom na mreži (Figure 2). Kada radni čvor postane nedostupan, Swarm njegove zadatke raspoređuje na druge čvorove [11]. Upravitelj vidi da rade dva kontejnera, a želi tri, i pokreće novi: novi kontejner, s novom adresom (**?@fig-swarm-reschedule**).

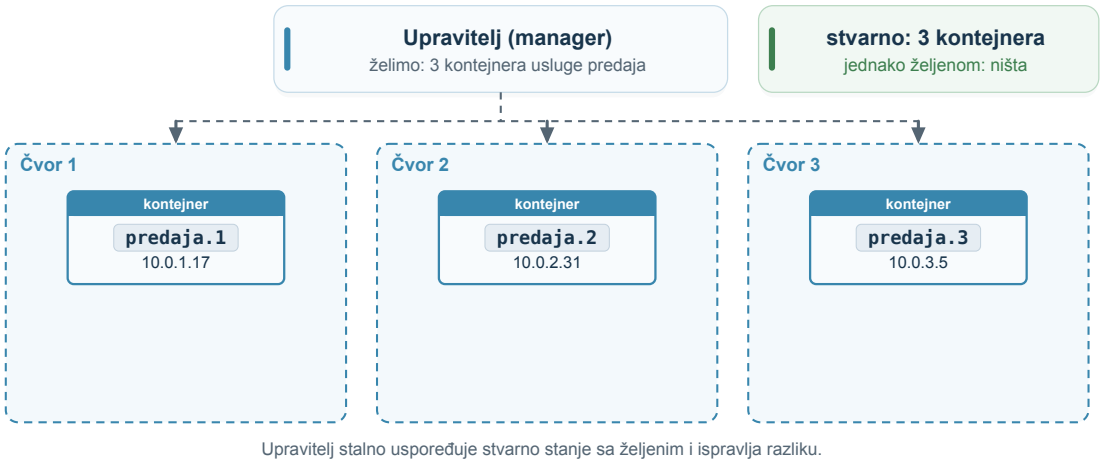
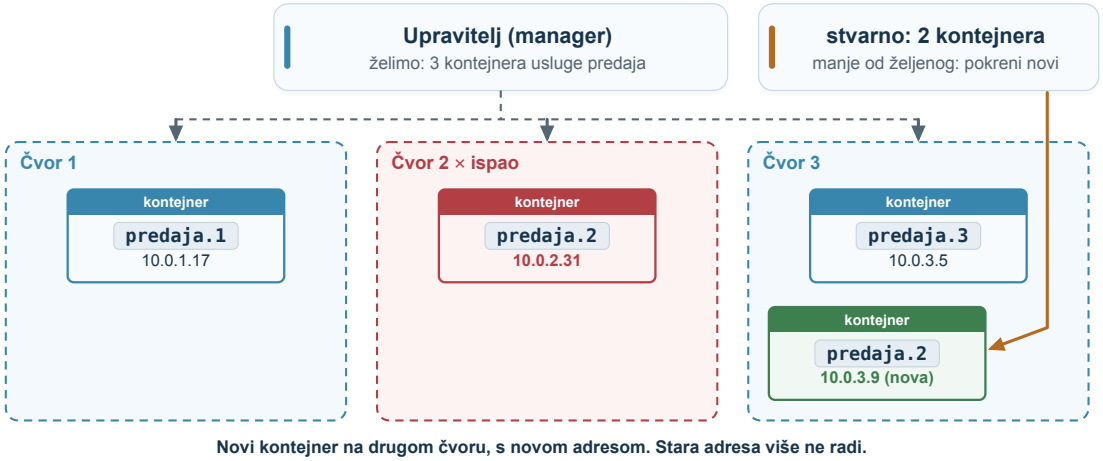


Figure 2: Upravitelj želi tri kontejnera usluge predaje i raspoređuje ih na tri čvora; stvarno stanje jednako je željenom. Adrese su ilustrativne.



Kada promet naraste, dodajemo replike; kada padne, uklanjamo ih. Skup adresa iza imena `predaja` stalno se mijenja, a portal o tome ne smije morati razmišljati.

Pitanje za ovo predavanje: kako portal treba pronaći uslugu predaje? Može zapisati IP adresu jedne instance u konfiguraciju. Može znati samo ime `predaja`, a trenutnu adresu dobiti od DNS-a, sustava koji imena pretvara u adrese. Ili može pitati poseban registar usluga: bazu u koju se instance same upisuju i iz koje portal čita popis instanci koje rade. Svaki odgovor vrijedi pod drukčijim pretpostavkama, a do kraja predavanja ćemo ih usporediti.

2.1 Za početak: dva uređaja i e-pošta

Predavanje 03 završilo je s dva slučaja. U prvome studentica pošalje ZIP s laptopa, ništa se ne događa, pa isti ZIP pošalje i s mobitela. Svaki preglednik generirao je vlastiti ključ idempotentnosti. Usluga predaje vidi dva različita ključa, dakle dvije različite operacije, i

snima dva zapisa iste predaje. Ako vrijedi jedna predaja po zadatku, nastavnik vidi dvije i ne zna koja vrijedi. Ključ generiran u pregledniku označava pokušaj s jednog uređaja, a ne predaju. Bolji je ključ koji opisuje samu predaju, na primjer student, zadatak i sažetak (hash) sadržaja ZIP-a: isti ZIP s bilo kojeg uređaja daje isti ključ, a nova verzija projekta ima drugi sažetak i ispravno postaje nova predaja. Druga je mogućnost da portal izda identifikator predaje prije prijenosa, pa ga oba uređaja koriste.

U drugom slučaju usluga pošalje e-poštu, a zatim se sruši prije nego što spremi ključ i zapis. Ponovni pokušaj snima predaju i ponovno šalje e-poštu. Transakcija u bazi ne pomaže jer e-pošta nije u bazi. Rješenje je u istoj transakciji zapisati i namjeru „pošalji e-poštu”, a slati je kasnije, iz baze (predavanja 09 i 10).

Oba slučaja vrte se oko istog pitanja: čime označavamo stvari tako da ih kasnije možemo prepoznati? To je tema današnjeg predavanja.

3 Ime, adresa i identifikator

Definicija 4.1

Ime (name) je niz znakova kojim se pozivamo na entitet: uslugu, računalo, datoteku ili korisnika. **Adresa (address)** je ime mjesta na kojemu entitet u ovom trenutku možemo kontaktirati, na primjer IP adresa i vrata. **Identifikator (identifier)** je ime koje označava točno jedan entitet i ne koristi se ponovno za drugi.

U našem portalu:

Primjer	Vrsta	Kada se mijenja?
`predaja`	ime usluge	nikad, osim ako uslugu sami preimenujemo
`10.0.1.17:50051`	adresa kontejnera	kad Swarm kontejner premjesti ili zamijeni novim
`a3f9`	ID kontejnera	nikad za taj kontejner; novi kontejner dobiva novi ID, a `a3f9` se ne daje drugom
`p-2026-0812`	identifikator predaje	nikad; ne ovisi o kontejnerima (ključ idempotentnosti iz 03)



Figure 3: Jedno ime usluge, više adresa koje se mijenjaju. Instanca s adresom 10.0.3.5 srušena je ili premještena; ime ostaje isto.

Razlika je praktična. Adresa kaže **gdje** je nešto sada, a ime kaže **što** tražimo. Ako portal u konfiguraciji drži adresu `10.0.1.17:50051`, svako premještanje instance traži promjenu konfiguracije portala, a portal zna samo za jednu instancu. Ako drži ime `predaja`, premještanje je posao sustava koji ime pretvara u adresu.

> **Definicija 4.2**
> **Razrješavanje imena (name resolution)** je pretvaranje imena u adresu ili druge podatke potrebne za pristup entitetu.

Ovo je lokacijska transparentnost iz predavanja 03, sada s druge strane: tamo je RPC skrivao da je funkcija udaljena, a ovdje ime skriva gdje se usluga nalazi. Ni ovdje se skrivanje ne može izvesti do kraja. Između premještanja usluge i trenutka kada klijent sazna novu adresu postoji razdoblje u kojemu klijent ima zastarjelu adresu.

4 DNS: imena na Internetu

Najpoznatiji sustav za razrješavanje imena je DNS (Domain Name System). Prostor imena DNS-a je **stablo** [1]: korijen, ispod njega vršne domene poput `hr` i `com`, ispod njih `unipu.hr` i tako dalje. Svako ime, poput `portal.unipu.hr`, put je od lista prema korijenu.

U DNS-u sudjeluju dvije vrste programa [1]. **Poslužitelji imena** (name servers) drže podatke o dijelu stabla. **Resolveri** su programi koji te podatke izvlače iz poslužitelja imena kada ih klijent zatraži. Kada portal želi adresu imena, pita resolver; resolver po potrebi pita više poslužitelja imena, od korijena prema dolje, dok ne dobije zapis s adresom.



Figure 4: Lijevo: prostor imena DNS-a je stablo. Desno: resolver pita poslužitelje imena redom, a odgovor sprema u predmemoriju do isteka TTL-a. Imena i koraci su ilustrativni.

Zapis koji povezuje ime s IPv4 adresom zove se zapis A. Za usluge postoji i **zapis SRV**, koji osim ciljnog računala nosi **vrata**, **prioritet** i **težinu** za odabir među više poslužitelja [3]. Oblik je `_usluga._protokol.ime TTL klasa SRV prioritet težina vrata cilj`, na primjer (ilustrativno):

```
_grpc._tcp.predaja.unipu.hr. 30 IN SRV 10 60 50051 cvor2.unipu.hr.
```

4.1 Predmemoriranje u DNS-u i TTL

Kada bi svaki poziv usluge prvo prošao cijeli put kroz stablo, DNS bi bio spor i preopterećen. Zato resolveri i klijenti odgovore **predmemoriraju** (cache): čuvaju ih neko vrijeme i koriste bez novog upita.

Definicija 4.3

TTL (time to live) zapisa u DNS-u određuje koliko dugo resolver ili klijent smije čuvati taj zapis u predmemoriji prije nego što ga treba odbaciti i ponovno pitati DNS [1].

TTL je kompromis. Dug TTL znači malo upita i brze odgovore, ali i dugo razdoblje u kojemu klijent može koristiti zastarjelu adresu. Kratak TTL znači da klijenti brzo saznaju promjene, ali šalju mnogo više upita.

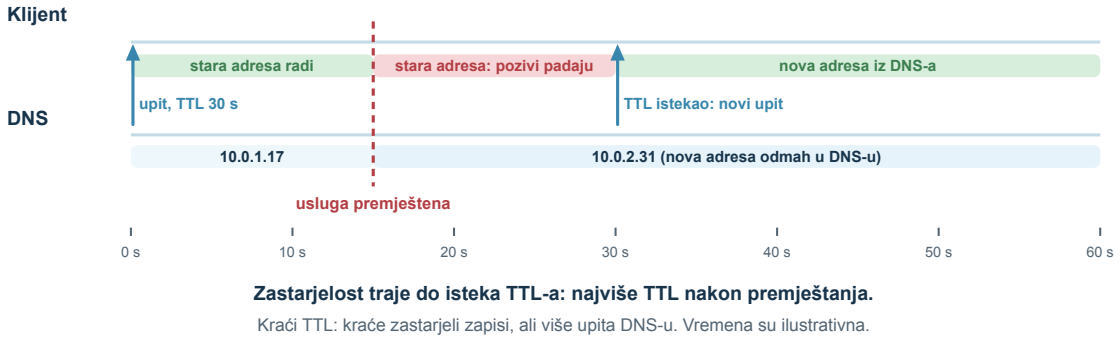


Figure 5: Klijent je adresu dobio u 0. sekundi s TTL-om od 30 s. Usluga se premjesti u 15. sekundi, a DNS odmah vraća novu adresu. Klijent ipak do 30. sekunde zove staru adresu. Vremena su ilustrativna.

Ako se usluga premjesti, a DNS odmah vrati novu adresu, klijent koji je staru adresu spremio neposredno prije premještanja koristi je još najviše jedan TTL. Ako klijenti adrese dohvaćaju u nasumičnim trenucima, prosječna zastarjelost je otprilike pola TTL-a; to je procjena uz tu pretpostavku, a ne jamstvo.

Predmemoriraju se i **negativni** odgovori. Ako resolver dobije odgovor da ime ne postoji (NXDOMAIN), taj odgovor treba spremiti, a njegovo trajanje uzima se iz zapisa SOA zone: manja od vrijednosti polja MINIMUM i TTL-a samog zapisa SOA [2, odjeljak 5]. Posljedica je česta zabuna pri pokretanju nove usluge: ako ju je klijent tražio minutu prije nego što je ime dodano, i dalje može dobivati odgovor „ne postoji”.

Klijent se može braniti i sam: kada poziv na predmemoriranu adresu ne uspije, može odbaciti zapis iz predmemorije i ponovno razriješiti ime. To skraćuje zastarjelost, ali opet traži da je ponovni pokušaj siguran, što je tema predavanja 03.

5 Imena usluga u kontejnerima

U predavanju 02 pokretali smo portal u Docker Composeu i Swarmu. Oba alata imaju ugrađeno razrješavanje imena usluga.

Docker Compose za projekt stvara mrežu `<ime-projekta>_default` i na nju spaja sve usluge; svaka usluga registrira svoje ime na internom DNS poslužitelju [4]. Usluga `portal` zato do usluge `predaja` dolazi preko imena, kao u dokumentaciji, gdje se usluga `web` spaja na bazu adresom `postgres://db:5432` [4]:

```
services:
  portal:
    image: portal
    environment:
      PREDAJA_ADRESA: "predaja:50051"   # ime usluge, ne IP adresa
```

```
  predaja:
    image: predaja
```

Docker Swarm nudi dva načina. Ako usluga objavljuje vrata, Docker joj dodjeljuje **virtualnu IP adresu** (VIP): klijent dobiva jednu adresu, a Swarm zahtjeve raspoređuje među replikama [5]. Druga mogućnost je **DNS round-robin** (`--endpoint-mode dnsrr`): DNS vraća adrese replika, a klijent bira sam [5]. U prvom slučaju klijent ne vidi replike, a u drugom ih vidi i mora se nositi s promjenama tog popisa.

Kubernetes rješava isti problem objektom Service. Podovi su prolazni i ne treba očekivati da je pojedini pod pouzdan i trajan; Deployment ih stvara i uklanja dinamički [6]. Service čini skup podova dostupnim na mreži, a klijenti ne bi trebali sami pratiti koji podovi trenutno čine taj skup; Kubernetes ažurira popis krajnjih točaka kad god se skup podova promijeni [6].

gRPC klijent iz predavanja 03 po zadanom koristi DNS za razrješavanje imena; adresa kanala može se zapisati i kao `dns:///predaja:50051` [7].

6 Otkrivanje usluga

DNS je razvijen za imena koja se rijetko mijenjaju. Instance usluga u klasteru pojavljuju se i nestaju svakih nekoliko minuta. Zato sustavi s mnogo usluga koriste **registar usluga** (service registry): bazu u kojoj se za svako ime usluge drži popis adresa instanci koje trenutno rade.

Definicija 4.4

Otkrivanje usluga (service discovery) je postupak kojim klijent za zadano ime usluge saznaje adrese instanci koje trenutno mogu obraditi zahtjev.

6.1 Na klijentu ili preko usmjernika

Richardson razlikuje dva obrasca [8]. Kod **otkrivanja na klijentu** (client-side discovery) klijent pita registar za adrese instanci, sam odabere jednu i pozove je. Kod **otkrivanja preko usmjernika** (server-side discovery) klijent šalje zahtjev usmjerniku (load balancer) na poznatoj adresi, a usmjernik pita registar i prosljeđuje zahtjev jednoj instanci [8].



Figure 6: Lijevo: portal pita registar i sam bira instancu. Desno: portal zna samo adresu usmjernika, a usmjernik dobiva adrese iz registra i bira instancu.

	Na klijentu	**Preko usmjernika**
Tko bira instancu	klijent	usmjernik
Što klijent mora znati	registar i pravila odabira	samo adresu usmjernika
Dodatni skok mreže	nema	svaki zahtjev prolazi kroz usmjernik
Primjer	Swarm s `dnsrr`, gRPC s popisom adresa	Swarm s VIP-om, Kubernetes Service

6.2 Tko upisuje instance u registar?

Registar vrijedi samo ako je točan. Kod **samoregistracije** (self-registration) instanca sama upisuje sebe u registar i obično mora povremeno obnavljati registraciju, kako bi registar znao da je još živa [9]. Kod **registracije treće strane** (third-party registration) to radi poseban registrator, na primjer platforma za razmještaj, koji može i provjeravati zdravlje instanci (health checks) [9].

Oba pristupa imaju isti problem: instanca se može srušiti bez odjave. Srušena instanca ne može obrisati vlastiti zapis. Potreban je mehanizam koji zapis uklanja sam, ako ga nitko ne potvrđuje.

7 Najam

Definicija 4.5

Najam (lease) je zapis ili pravo koje vrijedi ograničeno vrijeme (TTL) i nestaje ako ga vlasnik na vrijeme ne obnovi.

etcd, replicirana pohrana parova (ključ: vrijednost) iz predavanja 01, nudi upravo to. Klaster dodjeljuje najmove s ograničenim trajanjem; najam istječe ako klaster unutar TTL-a ne primi obnovu, a kada najam istekne ili se opozove, brišu se svi ključevi vezani uz njega [10]. Obnova se šalje pozivom `LeaseKeepAlive` [10].

Instanca usluge predaje tako se registrira ključem vezanim uz najam (naredbe prema dokumentaciji etcd-a; identifikator najma je ilustrativan):

```
etcdctl lease grant 10                          # najam s TTL-om od 10 s
etcdctl put --lease=694d7a1b /usluge/predaja/a3f9 "10.0.1.17:50051"
etcdctl lease keep-alive 694d7a1b               # obnova dok instanca radi
etcdctl get --prefix /usluge/predaja/           # otkrivanje: sve žive instance
```

Ako se instanca sruši, obnove prestanu, najam istekne i ključ nestane. Registar se čisti sam.

Ključevi su putanje, kao datoteke u mapama, pa registar izgleda kao grana stabla (Figure 7). To je dogovor o imenima: etcd ključeve drži u ravnom prostoru poredanih ključeva, a upit `--prefix` vraća sve ključeve s istim početkom [10]. Svaka instanca ima vlastiti ključ i vlastiti najam, pa istek jednog najma briše samo ključ te instance.

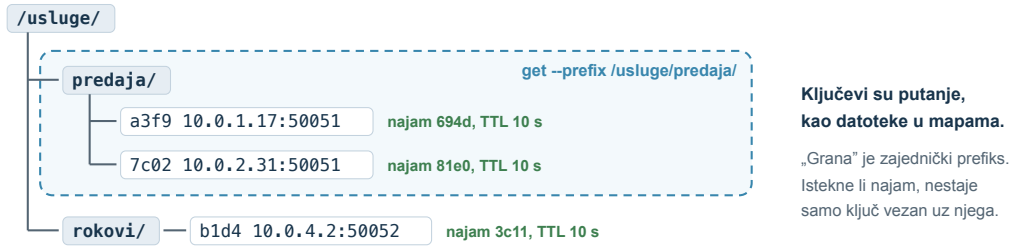


Figure 7: Ključevi registra u etcd-u kao stablo. Svaka instanca usluge predaje ima ključ pod /usluge/predaja/ vezan uz vlastiti najam; portal čita cijelu granu upitom get –prefix. Identifikatori i adrese su ilustrativni.

Za usporedbu, DynamoDB ima dva mehanizma koji zvuče slično, a ponašaju se vrlo različito. **TTL atribut** stavke označava trenutak nakon kojega stavka više nije potrebna; DynamoDB takve stavke briše sam, ali tek u roku od nekoliko dana nakon isteka, a do tada ih upiti vraćaju i treba ih filtrirati [13]. To je čišćenje starih podataka, a ne najam: srušena instanca

ostala bi u registru danima. Pravi najam nad DynamoDB-om gradi biblioteka **DynamoDB Lock Client**: vlasnik zaključavanja šalje **otkucaje**: pozadinska dretva stalno mijenja broj verzije zapisa (u primjeru iz dokumentacije najam traje 10 s, a otkucaj se šalje svake 3 s). Klijent koji želi isto zaključavanje zapamti broj verzije i pokrene štopericu; ako je broj nakon trajanja najma i dalje isti, zaključuje da je najam istekao i preuzima zaključavanje [14]. U zapisu se ne čuva apsolutno vrijeme, samo trajanje najma, pa satovi različitih računala ne moraju biti usklađeni (o satovima govori predavanje 05). Pitanje uvijek glasi isto: tko i kada briše zapis kojega nitko ne obnavlja?

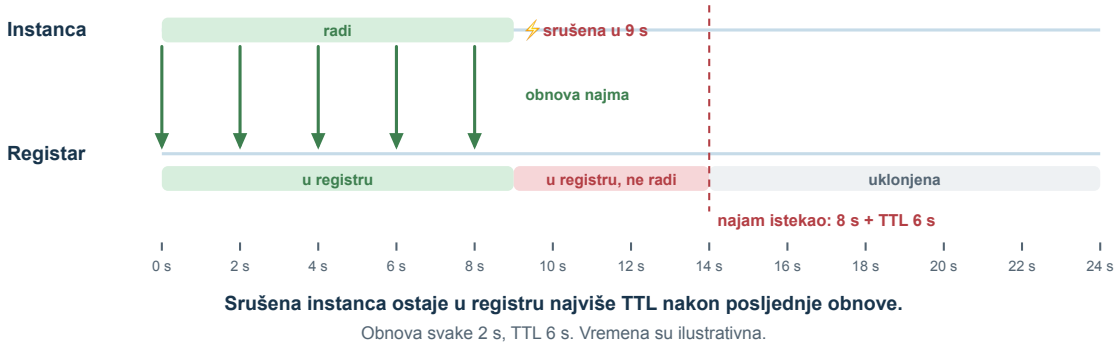


Figure 8: Instanca obnavlja najam svake 2 s, TTL je 6 s. Srušena u 9. sekundi, ostaje u registru do 14. sekunde: 6 s nakon posljednje obnove u 8. sekundi. Vremena su ilustrativna.

7.1 Koliko dug najam?

Najam ne uklanja srušenu instancu odmah. Ona ostaje u registru najviše jedan TTL nakon posljednje obnove, a za to vrijeme klijenti joj šalju zahtjeve koji ne uspijevaju. Kraći TTL to razdoblje skraćuje, ali obnove tada moraju biti češće, a to opterećuje registar.

Postoji i suprotna opasnost. Instanca koja privremeno zastane, na primjer zbog dugog skupljanja smeća (garbage collection) ili preopterećenog računala, ne šalje obnove iako nije srušena. Ako pauza traje dulje od TTL-a, registar je ukloni: to je **lažno uklanjanje**. Kada se instanca vrati, mora se ponovno registrirati s novim najmom. Isto se događa ako se obnova šalje prerijetko u odnosu na TTL. Razumno je obnovu slati nekoliko puta unutar jednog TTL-a, kako jedna izgubljena ili zakašnjela obnova ne bi izbacila ispravnu instancu.

To je ista dilema kao istek čekanja u predavanju 01: šutnja ne razlikuje sporu instancu od srušene. Najam ne rješava tu nesigurnost, nego je ograničava: registar se nakon TTL-a odluči, a sustav mora podnijeti da je odluka ponekad pogrešna. Detaljnije o otkrivanju ispada govori predavanje 08.

8 Natrag na početak: kako portal pronalazi uslugu predaje?

- **IP adresa u konfiguraciji:** radi samo dok se usluga ne premjesti i dok postoji jedna instanca. Za naš portal pred rok nije dovoljno.
- **Ime i DNS:** portal zove `predaja:50051`, a Compose ili Swarm razrješavaju ime. Zastarjelost je ograničena TTL-om predmemorije; Swarmov VIP je skriva iza jedne adrese.
- **Registar usluga s najmom:** instance se same registriraju i obnavljaju najam, a portal ili usmjernik čita žive instance. Srušena instanca ostaje u registru najviše jedan TTL.

U svakom slučaju portal mora podnijeti poziv na adresu koja više ne radi: s rokom čekanja, ponovnim razrješavanjem imena i sigurnim ponovnim pokušajem iz predavanja 03.

9 Kako napraviti loše nazivlje

- IP adrese zapisane u kodu ili konfiguraciji.
- Predmemorija bez TTL-a: adresa se dohvati jednom i čuva zauvijek.
- TTL od nekoliko sati za uslugu koja se premješta svaki dan.
- Registracija bez najma: srušene instance ostaju u registru zauvijek.
- Najam kraći od uobičajene pauze instance, ili obnova jednom po TTL-u.
- Klijent koji nakon neuspjeha ne razrješava ime ponovno.
- Ime koje opisuje mjesto umjesto uloge, na primjer `cvor2-predaja`.

10 Zadaci i rasprava

1. **Ime, adresa ili identifikator?** Razvrstajte i obrazložite: `predaja`, `10.0.2.31:50051`, `p-2026-0812`, `7c02` (ID kontejnera), `portal.unipu.hr`, `https://portal.unipu.hr/predaje/812`.
2. **TTL.** Zapis usluge ima TTL 300 s, a klijent poziva uslugu svake sekunde. Koliko najdulje klijent može zvati staru adresu nakon premještanja? Koliko upita DNS-u šalje u sat vremena s TTL-om 300 s, a koliko bez predmemorije?
3. **Negativna predmemorija.** Nova usluga `ocjene` dobila je ime u DNS-u u 10:00. Klijent ju je tražio u 9:59 i dobio odgovor da ime ne postoji. Zašto može i u 10:02 dobiti isti odgovor? O čemu ovisi koliko dugo?
4. **Najam.** TTL najma je 15 s, a instanca ga obnavlja svakih 5 s. Instanca se sruši 1 s nakon obnove. Koliko dugo ostaje u registru? Što se događa ako ispravna instanca zastane 20 s?
5. **Otkrivanje.** Za svaki slučaj odaberite otkrivanje na klijentu ili preko usmjernika i obrazložite: (a) preglednik studenta i portal; (b) portal i usluga predaje unutar klastera, preko gRPC-a.
6. **Swarm.** Portal poziva `predaja:50051`, a usluga predaje ima tri replike. Što portal dobiva od DNS-a ako usluga koristi VIP, a što ako koristi `dnsrr`? Tko u svakom slučaju bira repliku?

10.1 Predloženo rješenje

1. `predaja` je ime usluge. `10.0.2.31:50051` je adresa jedne instance. `p-2026-0812` i ID kontejnera `7c02` su identifikatori: svaki označava točno jedan entitet i ne koristi se ponovno. `portal.unipu.hr` je ime računala ili usluge u DNS-u. URL je ime resursa koje uključuje i način pristupa (protokol); njegov dio `portal.unipu.hr` treba razriješiti u adresu.
2. Najdulje oko 300 s, ako je klijent adresu dohvatio neposredno prije premještanja. S TTL-om 300 s klijent šalje oko 12 upita na sat; bez predmemorije jedan upit po pozivu, dakle oko 3600.
3. Resolver je predmemorirao negativan odgovor. Trajanje ovisi o zapisu SOA zone: manja od vrijednosti polja MINIMUM i TTL-a zapisa SOA.
4. Instanca ostaje u registru 15 s nakon posljednje obnove, dakle oko 14 s nakon rušenja. Pauza od 20 s dulja je od TTL-a: registar uklanja ispravnu instancu, a ona se nakon pauze mora ponovno registrirati.
5. (a) Preko usmjernika: preglednik zna samo javno ime portala, a raspodjela među instancama portala posao je usmjernika. (b) Oba su moguća. Na klijentu štedi skok mreže i omogućuje gRPC-u da sam bira instancu; preko usmjernika pojednostavljuje portal. Treba obrazložiti izbor.
6. S VIP-om DNS vraća jednu virtualnu adresu, a repliku bira Swarm. S `dnsrr` DNS vraća adrese replika, a repliku bira portal; portal mora podnijeti da se popis mijenja.

11 Sažetak

- Ime kaže što tražimo, adresa gdje je to sada, a identifikator označava točno jedan entitet.
- Razrješavanje imena pretvara ime u adresu. DNS to radi kroz stablo imena, resolvere i poslužitelje imena.
- Predmemoriranje ubrzava razrješavanje, a TTL ograničava koliko dugo zapis može biti zastario. Predmemoriraju se i negativni odgovori.
- Compose, Swarm, Kubernetes i gRPC daju uslugama imena; Swarm i Kubernetes mogu skrivati replike iza jedne adrese.
- Otkrivanje usluga koristi registar instanci; instancu bira klijent ili usmjernik.
- Najam čisti registar od srušenih instanci najviše jedan TTL nakon posljednje obnove, uz rizik lažnog uklanjanja ispravne instance.

11.1 Osmislite sljedeći korak: dva registra

Registar usluga je i sam usluga koja može ispasti. Pretpostavite da imate dvije kopije registra. Instanca se registrira u prvoj kopiji, a portal čita iz druge prije nego što se kopije usklade. Što portal vidi? Što ako se kopije zbog particije ne mogu uskladiti? Smije li registar odgovarati dok ne zna je li mu popis aktualan?

12 Izvori i dodatna literatura

1. P. Mockapetris, „Domain Names: Concepts and Facilities”, RFC 1034, studeni 1987. https://www.rfc-editor.org/rfc/rfc1034.html
2. M. Andrews, „Negative Caching of DNS Queries (DNS NCACHE)”, RFC 2308, ožujak 1998. https://www.rfc-editor.org/rfc/rfc2308.html
3. A. Gulbrandsen, P. Vixie i L. Esibov, „A DNS RR for specifying the location of services (DNS SRV)”, RFC 2782, veljača 2000. https://www.rfc-editor.org/rfc/rfc2782.html
4. Docker, „Networking in Compose”. https://docs.docker.com/compose/how-tos/networking/
5. Docker, „Manage swarm service networks”. https://docs.docker.com/engine/swarm/networking/
6. Kubernetes, „Service”. https://kubernetes.io/docs/concepts/services-networking/service/
7. gRPC, „Custom Name Resolution”. https://grpc.io/docs/guides/custom-name-resolution/
8. C. Richardson, „Pattern: Client-side service discovery” i „Pattern: Server-side service discovery”, microservices.io. https://microservices.io/patterns/client-side-discovery.html, https://microservices.io/patterns/server-side-discovery.html
9. C. Richardson, „Pattern: Self Registration” i „Pattern: 3rd Party Registration”, microservices.io. https://microservices.io/patterns/self-registration.html, https://microservices.io/patterns/3rd-party-registration.html
10. etcd, „etcd API”, odjeljak „Lease API”, verzija 3.6. https://etcd.io/docs/v3.6/learning/api/
11. Docker, „Swarm mode key concepts”. https://docs.docker.com/engine/swarm/key-concepts/
12. Docker, „What is Docker?”. https://docs.docker.com/get-started/docker-overview/
13. Amazon Web Services, „Using time to live (TTL) in DynamoDB”, DynamoDB Developer Guide. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html
14. AWS Labs, „Amazon DynamoDB Lock Client”, GitHub. https://github.com/awslabs/amazon-dynamodb-lock-client